Домашнее задание 1 · 90 минут

# Управление агентом

Сдаётся ссылкой на ваш клон репозитория; дедлайн — начало следующего занятия

Задача

Исправить реальный баг issue #7: задача с названием из одних пробелов остаётся в списке. Спека — без ИИ, реализация — агентом, проверка — ваша.

Веб-версия и стартер: olonho.github.io/agentic-class/homework.html

Задача · ISSUE #7

# Пустая задача после правки

Воспроизведение: открыть задачу → стереть название до пробелов → сохранить.

Фактически: задача остаётся в списке; фильтр «Открытые» показывает пустую строку.

Ожидалось: не сохранять и показать «Название не может быть пустым».

Границы: пробел, табуляция, неразрывный пробел U+00A0.

R2–R3 в issue не упомянуты: их нужно найти самому, проверив весь экран, а не только отчёт.

Подготовка · 5 минут

# Стартер: Vite + React + TypeScript + Vitest

```
npm install    # зависимости
npm test       # 2 стартовых теста, зелёные
npm run dev    # Vite, обычно http://localhost:5173
npm run build  # tsc --noEmit + production-сборка
```

Нужен Node.js 20+. Команды и правила для агента — в AGENTS.md.

Шаг 1 · 15 минут · без ИИ

# SPEC.md — пять обязательных разделов

1. Requirement — одно предложение от лица пользователя.
2. Acceptance tests — по одному на R1–R4.
3. Non-goals — что вы сознательно не делаете.
4. Files in scope — src/tasks.ts и src/App.tsx.
5. Definition of done — какие сигналы должны быть зелёными.

ИИ на этом шаге не используется — это условие зачёта.

Шаг 2 · 20 минут

# Тесты первыми — агент их не увидит

Пишете тесты сами, до работы агента: на R1, R2, R3.

Файл — src/tasks.test.ts или отдельный: пустое имя, края, унаследованная строка, счётчик.

Агент не получает тесты и не должен их читать.

R4: на каждое требование R1–R3 — минимум один тест. Зелёные стартовые 2/2 ничего не доказывают.

Шаг 3 · 20 минут

# Агент работает, вы ведёте лог

```
Прочитай SPEC.md и реализуй задачу точно по нему.
Меняй только src/tasks.ts и src/App.tsx.
После изменений запусти npm test и покажи результат.
```

Код не правите. Каждое вмешательство — строка в INTERVENTIONS.md:

время · что · зачем (например: 14:05 · переписал промпт · агент менял файлы вне scope).

Шаг 4 · 10 минут

# Проверяете сами, не по слову агента

```
mkdir -p evidence
npm test                    # должно быть зелёно
npm run build               # типы и сборка
git diff --stat             # только src/tasks.ts и src/App.tsx
npm run dev                 # скриншот в evidence/app.png
```

На скриншоте: «Открытых: 1», пустых строк нет. Если тесты не зелёные, шаг не зачтён.

Шаг 5 · 10 минут

# Красный прогон: три поломки, три блока

(а) убрать обрезание пробелов в createTask;

(б) убрать валидацию в renameTask;

(в) убрать фильтрацию унаследованной пустой строки (R2).

```
npm test 2>&1 | tee -a evidence/red.txt   # блок (а)
# восстановить код, затем так же блоки (б) и (в)
```

Иначе

Если после поломки тесты остались зелёными — ваш сигнал ничего не проверяет; исправьте тесты и повторите.

Изменение требования · после шага 5

# U+200B — тоже пустота

Название из одного символа U+200B (zero-width space) тоже считается пустым.

Обновите SPEC.md, тесты и реализацию тем же циклом: спека → агент → проверка → красный прогон.

Новая поломка — отдельный блок (г) в evidence/red.txt.

Это и есть проверка на «вариант»: первая реализация на String.trim() этот случай не ловит.

# Каждый провал — в таксономию

Шаг 6 · 10 минут

## Планирование

спека

## Контекст

агент не знал

## Инструмент

команда

## Политика

перила

## Артефакт

не тот файл

## Проверка

ложный сигнал

## UX

неудобно

## Восстановление

откат

Шаг 6 · продолжение

# Одно правило, которое ловит тот же провал

Строка в REPORT.md: <категория>: <что произошло> → <правило>, с уликой (транскрипт, хунк диффа или имя теста).

Правило — один из трёх видов: тест, правило линтера или строка в AGENTS.md.

Оно должно ловить вариант провала, а не только увиденный случай.

Проверка: сломайте код так же, как в шаге 5 — правило обязано сработать.

Шаг 7 · 5 минут

# REFLECTION.md — пять строк, без ИИ

1. Что агент понял не так, как вы, и какой строкой спеки это было вызвано?
2. Какой ваш сигнал оказался ложным и что вы им заменили?
3. Какое правило вы добавили и какой провал оно закрывает?

Ровно пять строк. Без ИИ.

Сдача · 6 баллов

# Что лежит в репозитории

SPEC.md — 5 разделов, написан без ИИ · 1 балл

Тесты на R1–R3 в репозитории · 1 балл

evidence/app.png и evidence/red.txt с блоками (а)(б)(в) · 1 балл

INTERVENTIONS.md — все вмешательства · 1 балл

REPORT.md — таксономия и правило, которое ловит вариант · 1 балл

REFLECTION.md — пять строк · 1 балл

Скрытый набор тестов преподавателя — обязательный гейт: не проходит — не зачтено.

Защита 3 минуты: сломать одно из трёх мест, показать красное и объяснить, почему зелёный значит «проверено».

Бонус · +1

# Два прогона одной задачи: с SPEC и без

Прогон 1: агент получает задачу одним предложением (SPEC не показываете).

Прогон 2: тот же промпт, но с SPEC.md — как в шаге 3.

Таблица: время до первой сдачи · ходы · вмешательства · дефекты при приёмке.

Вывод одной фразой: что именно дала спецификация.

Без спецификации начинайте с чистого репозитория (git stash), иначе прогоны несравнимы.

Не зачтено, если...

# Стоп-условия

Стоп-условия

SPEC писал ИИ · нет трёх красных блоков · правило не срабатывает на своём провале · скрытый набор не проходит.

**Сдаётся портфолио процесса, а не патч агента.**

Оценивается то, что вы проверили сами, и правило, которое не даст провалу повториться.